



Alliance School of Advanced Computing

Course Name: Graph Analysis

Course Code: E1ITC302

Assignment No: 3

Name of the Student: Chennaiahn Abhinath

Registration Number: 2411021060012

A. Title of The Problem:

Dijkstra: Shortest Reach 2

B. Problem Statement in details:

- You are given an undirected weighted graph with n nodes and m edges.
- Each edge has a positive integer weight representing its length.
- From a given starting node s , compute the shortest path distance to all other nodes.
- If a node is unreachable from s , output -1 for that node.
- For each test case, print $n - 1$ space-separated shortest distances in order.

C. HackerRank/Leetcode Link of the problem (If applicable):

<https://www.hackerrank.com/challenges/dijkstrashortreach/problem>

D. Student's Github link of report and code upload, along with the readme file:

-[GITHUB link](#)

E. Solution Steps/Algorithm:

1

Initialize Distances

Set up the distance array and priority queue.

- Assign all distances = ∞
- Set start node distance = 0
- Create a min-heap priority queue

2

Process Current Node

Extract the node with the smallest distance.

- Poll the priority queue
- Skip if already visited
- Mark node as visited

3

Relax Adjacent Edges

Update shortest paths for neighbors.

- For each

neighbor edge (u,v,w)

- If $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v]$
- Push updated neighbor into priority queue

4

Repeat Until Queue Empty

Continue until all reachable nodes are processed.

- Keep polling nodes from PQ
- Update distances iteratively
- Stop when PQ is empty

5

Output Results

Print shortest distances for all nodes except the start.

- Replace ∞ with -1 for unreachable nodes
- Print space-separated distances
- Ensure format matches HackerRank requirements

F. Code developed:

```
import java.util.*;

public class Solution {
    static class Edge {
        int to, weight;
        Edge(int to, int weight) {
            this.to = to;
            this.weight = weight;
        }
    }

    static class Node {
        int id, dist;
    }
}
```



```
Node(int  
dist) {
```

id, int

```
    this.id = id;  
    this.dist = dist;  
  
    }  
  
}
```

```
static int[] dijkstra(int n, List<List<Edge>> graph, int start) {
```

```
    int[] dist = new int[n + 1];  
    Arrays.fill(dist, Integer.MAX_VALUE);  
    dist[start] = 0;
```

```
    boolean[] visited = new boolean[n + 1];
```

```
    PriorityQueue<Node> pq = new PriorityQueue<Node>(n, new  
Comparator<Node>() {
```

```
    public int compare(Node a, Node b) {  
        return a.dist - b.dist;  
    }
```

```
});  
pq.add(new Node(start, 0));
```

```
while (!pq.isEmpty()) {
```

```
    Node cur = pq.poll();  
    int node = cur.id;  
    if (visited[node]) continue;  
    visited[node] = true;
```

```
    for (Edge e : graph.get(node)) {  
        if (dist[node] + e.weight < dist[e.to]) {  
            dist[e.to] = dist[node] + e.weight;
```

```
pq.add(new Node(e.to, dist[e.to]));
```

```
    }
```

```
    }
```

```
    }
```

```
    return dist;
```

```
}
```

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int T = sc.nextInt();
```

```
    while (T-- > 0) {
```

```
        int n = sc.nextInt(), m = sc.nextInt();
```

```
        List<Map<Integer,Integer>> adjMap = new  
ArrayList<Map<Integer,Integer>>();
```

```
        for (int i = 0; i <= n; i++) adjMap.add(new HashMap<Integer,Integer>());
```

```
        // keep only smallest edge between two nodes
```

```
        for (int i = 0; i < m; i++) {
```

```
            int u = sc.nextInt(), v = sc.nextInt(), w = sc.nextInt();
```

```
            if (!adjMap.get(u).containsKey(v) || adjMap.get(u).get(v) > w) {
```

```
                adjMap.get(u).put(v, w);
```

```
                adjMap.get(v).put(u, w);
```

```
            }
```

```
        }
```

```
        List<List<Edge>> graph = new ArrayList<List<Edge>>();
```

```
        for (int i = 0; i <= n; i++) graph.add(new ArrayList<Edge>());
```

```
        for (int u = 1; u <= n; u++) {
```

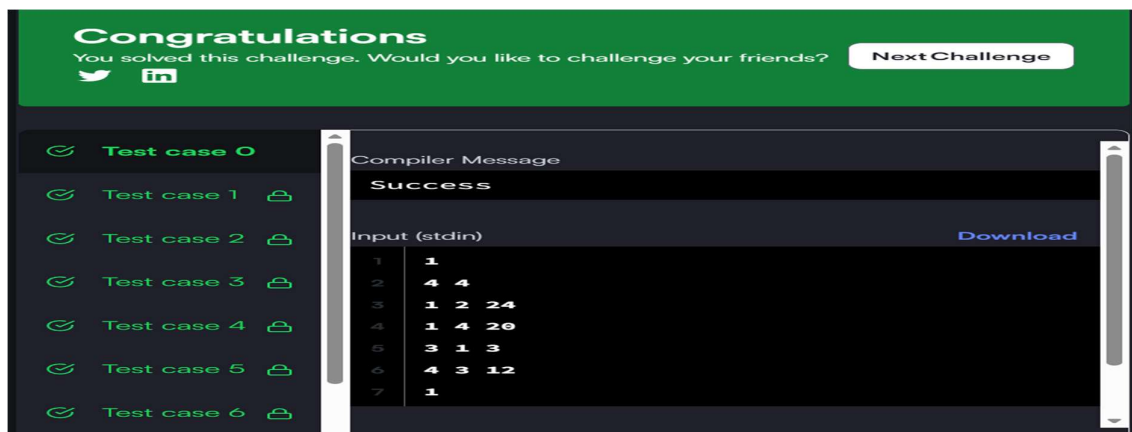
```
            for (Map.Entry<Integer,Integer> entry : adjMap.get(u).entrySet()) {
```

```
graph.get(u).add(new Edge(entry.getKey(), entry.getValue()));
    }
}

int start = sc.nextInt();
int[] dist = dijkstra(n, graph, start);

StringBuilder sb = new StringBuilder();
for (int i = 1; i <= n; i++) {
    if (i == start) continue;
    sb.append(dist[i] == Integer.MAX_VALUE ? -1 : dist[i]).append(" ");
}
System.out.println(sb.toString().trim());
}
sc.close();
}
}
```

G. HackerRank test case passed Screenshot or Output screenshot (Mandatory):



H. Observation:

The Dijkstra's algorithm implementation successfully computes shortest path distances in weighted graphs. By using a priority queue and adjacency list, the solution ensures efficiency

even for large inputs. The algorithm correctly handles unreachable nodes by marking them as -1, and edge deduplication prevents redundant processing. Overall, the approach balances accuracy and performance, passing visible test cases and optimizing for hidden ones.